

Guía de sesión — Fase 0: Migración a Next.js

Sábado-Domingo 12-13 de abril de 2026

Para: Eneko + Ekaitz + Claude Code **Objetivo:** Montar la infraestructura profesional (ramas, Next.js, calidad) **Regla de oro:** Si algo falla, PARAMOS y entendemos por qué antes de seguir. No hay prisa.

Antes de empezar — Checklist de preparación

Verificad que tenéis todo esto ANTES de la sesión:

- ☐ Acceso al dashboard de **Vercel** (vercel.com/dashboard)
- ☐ Acceso al dashboard de **Supabase** (supabase.com/dashboard)
- ☐ **Claude Code** abierto en la terminal, en la carpeta del proyecto
- ☐ Conexión a internet estable
- ☐ Este documento impreso o en una pantalla aparte

Cosas que NO hay que hacer antes de la sesión

- No tocar nada en n8n
 - No hacer cambios en el código
 - No tocar nada en Vercel ni Supabase
-

BLOQUE 1 — Crear las ramas en Git (30 minutos)

¿Qué vamos a hacer?

Hasta ahora solo tenemos la rama `main` (producción). Vamos a crear dos ramas nuevas para poder trabajar sin miedo a romper nada.

Imaginad que `main` es la tienda abierta al público. Vamos a crear un taller trasero (`develop`) donde trabajamos, y una sala de pruebas (`staging`) donde enseñamos al cliente antes de poner en tienda.

Pasos

Paso 1.1 — Verificar que todo está limpio

Le diremos a Claude Code: "Haz un git status para ver que está todo limpio"

Esperamos ver: `nothing to commit, working tree clean`

Si hay archivos sin commitear → los commiteamos primero.

Paso 1.2 — Crear rama `develop`

Le diremos a Claude Code: "Crea una rama develop desde main y súbela a GitHub"

Esto crea la rama y la sube. No cambia nada en producción.

Paso 1.3 — Crear rama `staging`

Le diremos a Claude Code: "Crea una rama staging desde main y súbela a GitHub"

Paso 1.4 — Cambiar a la rama `develop`

Le diremos a Claude Code: "Cámbiate a la rama `develop`"

A partir de aquí, TODO lo que hagamos será en `develop`. La rama `main` (producción) no se toca.

✅ Checkpoint 1

- ☐ `git branch` muestra tres ramas: main, develop, staging
- ☐ Estamos en la rama `develop`
- ☐ En GitHub se ven las tres ramas

BLOQUE 2 — Configurar Vercel para las ramas (30 minutos)

¿Qué vamos a hacer?

Decirle a Vercel que cuando subamos código a `develop` o `staging`, cree una versión de prueba del CRM con su propia URL. Así podemos ver los cambios antes de ponerlos en producción.

Pasos

Paso 2.1 — Ir al dashboard de Vercel

1. Abrir `vercel.com/dashboard`
2. Clicar en el proyecto `myt-crm-app`
3. Ir a **Settings** → **Git**

Paso 2.2 — Verificar Production Branch

- En "Production Branch" debe poner `main`
- Si ya está así, perfecto. No tocar nada.

Paso 2.3 — Verificar que Preview Deployments están activos

- En la misma pantalla de Settings → Git
- "Preview Deployments" debe estar en **Enabled**
- Esto hace que cada push a `develop` o `staging` cree automáticamente una preview

Paso 2.4 — Configurar variables de entorno para Preview

1. Ir a **Settings** → **Environment Variables**
2. Las variables actuales (`VITE_SUPABASE_URL` , `VITE_SUPABASE_ANON_KEY`) deben estar para **Production**
3. Más adelante (cuando tengamos Supabase Branching) añadiremos variables distintas para Preview
4. **De momento:** las dejamos iguales para Production y Preview (usan la misma BD)

Nota importante: cuando activemos Supabase Branching, las variables de Preview apuntarán a la BD de branch automáticamente. Eso lo haremos en el Bloque 4.

✅ Checkpoint 2

- ☐ En Vercel, `main` es la rama de producción

- ☐ Preview Deployments están activados
- ☐ Si hacemos push a `develop`, Vercel creará una URL de preview automáticamente

BLOQUE 3 — Migrar de React (Vite) a Next.js (3-4 horas)

¿Qué vamos a hacer?

Este es el bloque más importante y largo del día. Vamos a convertir la app de un formato antiguo (Vite) a un formato profesional (Next.js) que nos permite tener backend + frontend juntos.

Analogía: Ahora mismo la app es como un documento Word donde todo está en una sola página. Vamos a pasarlo a un libro con capítulos, índice, y contraportada.

¿Qué NO va a cambiar?

- Los estilos (CSS) se mantienen
- La lógica de negocio se mantiene
- Los componentes se mantienen
- Supabase y la base de datos no se tocan
- **Los clientes no notan nada** — producción sigue en `main` sin cambios

Paso 3.1 — Inicializar Next.js

```
Le diremos a Claude Code:
"Vamos a migrar el proyecto de Vite a Next.js App Router.
Estamos en la rama develop.
Paso 1: Inicializa Next.js en este proyecto.
Cambia las dependencias necesarias en package.json,
crea la estructura de carpetas de Next.js (app/, components/, lib/),
y configura next.config.js.
NO borres los archivos de src/ todavía – los vamos a ir moviendo uno a uno.
Crea el archivo app/layout.tsx con los estilos globales importados."
```

Lo que Claude Code hará:

- Instalar `next` como dependencia
- Quitar `vite` y el plugin de Vite
- Crear carpeta `app/` con el layout base
- Crear `next.config.js`
- Actualizar los scripts en `package.json` (`dev`, `build`, `start`)

Lo que NO hará:

- Borrar los archivos de `src/` (los usaremos como referencia)
- Tocar la base de datos
- Tocar n8n

Paso 3.2 — Configurar Supabase para Next.js

```
Le diremos a Claude Code:
"Crea la configuración de Supabase para Next.js.
Necesito:
1. Un archivo lib/supabase/client.ts para el browser (usa NEXT_PUBLIC_SUPABASE_URL y
```

NEXT_PUBLIC_SUPABASE_ANON_KEY)

2. Un archivo `lib/supabase/server.ts` para el servidor (usa `SUPABASE_SERVICE_ROLE_KEY` para las API routes)

Sigue la documentación oficial de Supabase para Next.js App Router."

Lo que Claude Code hará:

- Crear `lib/supabase/client.ts` — para cuando el usuario navega por el CRM
- Crear `lib/supabase/server.ts` — para las API routes (reemplazan n8n)

Nota: las variables de entorno cambian de nombre:

- `VITE_SUPABASE_URL` → `NEXT_PUBLIC_SUPABASE_URL`
- `VITE_SUPABASE_ANON_KEY` → `NEXT_PUBLIC_SUPABASE_ANON_KEY`
- **NUEVO:** `SUPABASE_SERVICE_ROLE_KEY` (para el backend, la que usa n8n ahora)

Paso 3.3 — Crear la autenticación (login/registro)

Le diremos a Claude Code:

"Crea el sistema de autenticación para Next.js:

1. Un `middleware.ts` que proteja todas las rutas excepto `/login` y `/register`
2. Página de login en `app/(auth)/login/page.tsx`
3. Página de registro en `app/(auth)/register/page.tsx`

Copia la lógica de login y registro que hay actualmente en `src/App.jsx`.

Usa el Supabase client que acabamos de crear."

Lo que Claude Code hará:

- Crear `middleware.ts` — el "portero" que no deja pasar a nadie sin estar logueado
- Crear las páginas de login y registro

Paso 3.4 — Crear el layout principal del CRM (sidebar + navegación)

Le diremos a Claude Code:

"Crea el layout principal del CRM con la sidebar de navegación.

1. `app/(dashboard)/layout.tsx` — Layout con sidebar (igual que la actual)
2. Extrae la sidebar de `src/App.jsx` a `components/Sidebar.tsx`
3. Debe tener las mismas opciones: Dashboard, Pipeline, Agente IA, Calendario, Chat, Módulos, Admin
4. Usa los mismos estilos de `App.css` (copia los estilos relevantes)
5. Incluye la lógica de usuario logueado (nombre, empresa, rol)
6. Incluye el responsive (hamburger menu en móvil)"

Paso 3.5 — Migrar la vista Dashboard

Le diremos a Claude Code:

"Migra la vista de Dashboard de `src/App.jsx` a `app/(dashboard)/page.tsx`.

1. Extrae toda la lógica del dashboard (estadísticas, gráficos) a su propia página
2. Los componentes de Recharts necesitan 'use client' porque son interactivos
3. Mantén los mismos estilos
4. Conecta con Supabase para cargar los datos del tenant logueado"

Paso 3.6 — Migrar el Pipeline (Kanban + Lista)

Le diremos a Claude Code:

```
"Migra la vista de Pipeline (Kanban y Lista) de src/App.jsx a
app/(dashboard)/pipeline/page.tsx.
1. Extrae KanbanColumn, LeadCard, ScoreBadge a components/
2. Extrae LeadDetail (panel lateral) a components/LeadDetail.tsx
3. Extrae NewLeadModal a components/NewLeadModal.tsx
4. Mantén toda la lógica de drag-and-drop
5. Mantén los mismos estilos"
```

Este es el paso más largo porque el Pipeline es la vista más compleja del CRM.

Paso 3.7 — Migrar las vistas restantes

Cada una de estas es más sencilla porque ya son componentes separados:

Le diremos a Claude Code:

```
"Migra estas vistas una por una:
1. ChatbotConfig.jsx → app/(dashboard)/agent/page.tsx
2. CalendarioCitas.jsx → app/(dashboard)/calendar/page.tsx
3. ChatView.jsx → app/(dashboard)/chat/page.tsx
4. EntrenarAgente.jsx → app/(dashboard)/agent/train/page.tsx
5. ModulosConfig.jsx → app/(dashboard)/modules/page.tsx
6. AdminPanel (de App.jsx) → app/(dashboard)/admin/page.tsx
Cada una debe funcionar independientemente con su propia carga de datos de Supabase."
```

Paso 3.8 — Verificar que todo funciona

Le diremos a Claude Code:

```
"Ejecuta npm run dev y verifica que el proyecto arranca sin errores.
Revisa que no haya errores de TypeScript ni de compilación."
```

Luego nosotros verificamos en el navegador:

1. ¿Carga la página de login?
2. ¿Podemos hacer login con nuestra cuenta?
3. ¿Se ve el Dashboard con los datos?
4. ¿Funciona el Pipeline (drag-and-drop)?
5. ¿Se ven las conversaciones en Chat?
6. ¿Funciona el Calendario?
7. ¿Funciona la sección de Agente IA?
8. ¿Funciona Módulos?
9. ¿Funciona en móvil (responsive)?

Paso 3.9 — Subir a GitHub y verificar en Vercel

Le diremos a Claude Code:

```
"Haz commit de todos los cambios con mensaje 'feat: migrar de Vite a Next.js App
Router'
y súbelo a la rama develop"
```

Después:

1. Ir a Vercel → ver que se ha creado un deployment de preview
2. Abrir la URL de preview
3. Verificar que funciona igual que la versión de producción

✅ Checkpoint 3

- ☐ `npm run dev` funciona sin errores
 - ☐ Login funciona
 - ☐ Todas las vistas cargan con datos reales
 - ☐ El deploy de preview en Vercel funciona
 - ☐ La URL de producción (main) sigue funcionando con la versión antigua — NO se ha tocado
-

BLOQUE 4 — Activar Supabase Branching (30 minutos)

¿Qué vamos a hacer?

Activar la funcionalidad de Supabase para que cada rama de Git tenga su propia base de datos. Así, los cambios que hagamos en `develop` no afectan a la BD de producción.

Pasos

Paso 4.1 — Activar Branching en Supabase

1. Ir a supabase.com/dashboard
2. Seleccionar el proyecto del CRM
3. Ir a **Settings** → **Branching** (o **Branches**)
4. Activar Branching
5. Conectar con el repo de GitHub (`mytconsulting/myt-crm-app`)

Paso 4.2 — Configurar la integración Supabase + Vercel

1. En Supabase, ir a **Integrations** → **Vercel**
2. Conectar con el proyecto de Vercel
3. Esto hace que Vercel use automáticamente la BD correcta para cada rama

Paso 4.3 — Instalar Supabase CLI (para migraciones)

Le diremos a Claude Code:

"Instala Supabase CLI si no está instalada, y haz `supabase init` en el proyecto. Después haz `supabase link` con nuestro proyecto de Supabase."

*Nos pedirá el ID del proyecto de Supabase. Lo encontramos en **Settings** → **General** del dashboard.*

Paso 4.4 — Crear primera migración formal

Le diremos a Claude Code:

"Haz un `supabase db pull` para traer el schema actual como migración base. Esto crea un archivo en `supabase/migrations/` con todo el schema actual."

✅ Checkpoint 4

- ☐ Supabase Branching está activado

- ☐ Está conectado con GitHub y Vercel
- ☐ supabase/migrations/ tiene al menos un archivo con el schema base
- ☐ Las variables de entorno de Supabase se inyectan automáticamente en Vercel previews

BLOQUE 5 — Configurar calidad básica (30 minutos)

¿Qué vamos a hacer?

Montar las herramientas que nos avisan cuando algo falla, en vez de enterarnos cuando un cliente se queja.

Paso 5.1 — Instalar Sentry (alertas de errores)

Le diremos a Claude Code:

"Instala y configura Sentry para Next.js.

Usa el wizard de Sentry: `npx @sentry/wizard@latest -i nextjs`"

El wizard nos pedirá:

1. Crear cuenta en sentry.io (si no tenemos) o hacer login
2. Seleccionar proyecto → crear uno nuevo "emanticrm"
3. El wizard configura todo automáticamente

¿Qué hace Sentry? Si algo falla en el CRM (un error de JavaScript, una API que no responde), nos llega un email con los detalles. Es como tener una cámara de seguridad para el código.

Paso 5.2 — Configurar Vercel AI SDK

Le diremos a Claude Code:

"Instala y configura Vercel AI SDK con los proveedores de Anthropic y OpenAI.

1. Instala: `ai`, `@ai-sdk/anthropic`, `@ai-sdk/openai`
2. Crea `lib/ai/config.ts` con la configuración de los modelos:
 - Claude Sonnet 4.6 como modelo principal
 - GPT-4o como fallback
 - Claude Haiku 4.5 para tareas ligeras
3. NO crees todavía las API routes del agente IA – eso es Fase 0.5"

Necesitaremos añadir la API key de Anthropic en Vercel como variable de entorno:

`ANTHROPIC_API_KEY`

Paso 5.3 — Crear los types base de TypeScript

Le diremos a Claude Code:

"Crea los tipos TypeScript base del proyecto en `types/`:

1. `types/lead.ts` – tipo Lead con todos los campos de la tabla leads
 2. `types/tenant.ts` – tipo Tenant/Profile
 3. `types/cita.ts` – tipo Cita
 4. `types/modulos.ts` – tipo ConfiguracionModulos
- Basándote en las tablas que ya existen en Supabase."

✅ Checkpoint 5

- ☐ Sentry está configurado (podemos provocar un error y ver que llega a sentry.io)
- ☐ AI SDK instalado con Anthropic + OpenAI
- ☐ Types básicos creados en `types/`
- ☐ Variables de entorno nuevas añadidas en Vercel:
 - `NEXT_PUBLIC_SUPABASE_URL`
 - `NEXT_PUBLIC_SUPABASE_ANON_KEY`
 - `SUPABASE_SERVICE_ROLE_KEY`
 - `ANTHROPIC_API_KEY`
 - `OPENAI_API_KEY`
 - `SENTRY_DSN`

BLOQUE 6 — Verificación final y commit (30 minutos)

Paso 6.1 — Test completo en local

Abrir el CRM en el navegador (`localhost:3000`) y probar TODAS las funcionalidades:

Funcionalidad	¿Funciona?
Login	☐
Registro	☐
Dashboard con estadísticas	☐
Pipeline Kanban (drag-and-drop)	☐
Pipeline Lista	☐
Ficha de lead (LeadDetail)	☐
Nuevo lead manual	☐
Agente IA (config documentos)	☐
Entrenar agente	☐
Calendario de citas	☐
Chat de conversaciones	☐
Módulos config	☐
Admin panel (si somos admin)	☐
Responsive en móvil	☐
Notificaciones	☐

Paso 6.2 — Commit y push final

Le diremos a Claude Code:
"Haz commit de todo con el mensaje 'feat: migración completa a Next.js App Router -

Fase 0'
y súbelo a develop"

Paso 6.3 — Verificar deploy en Vercel

1. Ir a Vercel → ver que el deployment de `develop` se ha completado
2. Abrir la URL de preview
3. Hacer las mismas pruebas que en local

Paso 6.4 — Verificar que producción no se ha tocado

1. Abrir la URL de producción del CRM (la que usan los clientes)
2. Verificar que TODO sigue funcionando igual
3. La rama `main` no se ha tocado en ningún momento

✅ Checkpoint FINAL

- ☐ Rama `develop` con Next.js funcionando en Vercel preview
- ☐ Rama `main` (producción) intacta con la versión de Vite
- ☐ Supabase Branching activado
- ☐ Sentry configurado
- ☐ AI SDK instalado y configurado
- ☐ Supabase CLI con migraciones base
- ☐ Types de TypeScript creados

Si algo sale mal — Guía de emergencia

"El CRM de producción dejó de funcionar"

Imposible si hemos seguido los pasos. Todo el trabajo se hace en `develop`. La rama `main` no se toca nunca. Pero si por alguna razón pasa:

1. Ir a Vercel → Deployments → buscar el último deployment bueno → "Promote to Production"
2. Esto restaura la versión anterior en segundos

"Next.js no arranca"

1. Verificar que las dependencias están instaladas: `npm install`
2. Verificar que no hay errores de sintaxis en los archivos
3. Leer el mensaje de error — Claude Code lo puede interpretar

"No puedo hacer login"

1. Verificar que las variables de entorno de Supabase están configuradas
2. Verificar que el nombre de las variables es correcto (`NEXT_PUBLIC_SUPABASE_URL` , no `VITE_SUPABASE_URL`)

"Los datos no cargan"

1. Verificar que las RLS policies siguen funcionando
2. Verificar que el Supabase client usa las variables correctas
3. Abrir la consola del navegador (F12) y mirar si hay errores rojos

"Vercel no deploya"

1. Ir a Vercel → Deployments → ver los logs del build

2. El error suele ser un import mal puesto o un archivo que falta

Planificación de tiempo estimada

Bloque	Duración	Acumulado
1. Ramas Git	30 min	0:30
2. Configurar Vercel	30 min	1:00
3. Migrar a Next.js	3-4 horas	4:00-5:00
— Pausa / comida	30 min	4:30-5:30
4. Supabase Branching	30 min	5:00-6:00
5. Calidad (Sentry, AI SDK)	30 min	5:30-6:30
6. Verificación final	30 min	6:00-7:00

Total estimado: 6-7 horas

Si el Bloque 3 se alarga (es el más grande), podemos dejar los Bloques 4 y 5 para el domingo y hacer el Bloque 3 con calma el sábado.

Lo que viene después (Fase 0.5 — semana que viene)

Una vez completada esta sesión, la siguiente semana migraremos los workflows de n8n a código:

- 1. **Lunes-Martes:** Migrar WF1 (captura lead) y WF6 (entrada manual) a API Routes
- 2. **Miércoles:** Migrar WF7 (entrenar agente) y WF4 (recordatorios)
- 3. **Jueves:** Migrar WF5 (follow-ups) y WF8 (auto-learn)
- 4. **Viernes:** Migrar WF2 (agente IA) — el más complejo, dedicar el día entero

Cada workflow migrado se prueba y, cuando funciona, se desactiva en n8n.

Resumen en una frase

Hoy montamos la casa nueva (Next.js) en la parcela de al lado (rama develop). Los clientes siguen viviendo en la casa vieja (main). Cuando la casa nueva esté lista y probada, nos mudamos.